Redis and the background worker

How SalesPal sends emails and builds reports without making you wait, explained in plain language.

1. The one-minute picture

Think of a restaurant. The **waiter** (the Next.js web app) takes your order and must come back to you quickly. He does not cook. He writes the order on a **ticket** and clips it on the **ticket rail** (Redis). The **cook** (the worker) takes tickets from the rail one at a time and does the slow work: sending the email, drawing the PDF.

The ticket only says *which* job to do (for example: email number 7). The full details of the email are written in the restaurant's **record book** (the Postgres database), which is where the screens read the result from.

Restaurant	SalesPal	Real thing in the project
Waiter	Web app	`npm start` (Next.js): routes in `app/api/...`
Ticket rail	Redis	A separate service. Keys start with `bull:`
Ticket	Job	Small record: `{ "emailJobId": 7 }`
Cook	Worker	`worker/index.mts` running as its own process
Record book	Postgres	Tables `email_jobs` and `report_jobs`

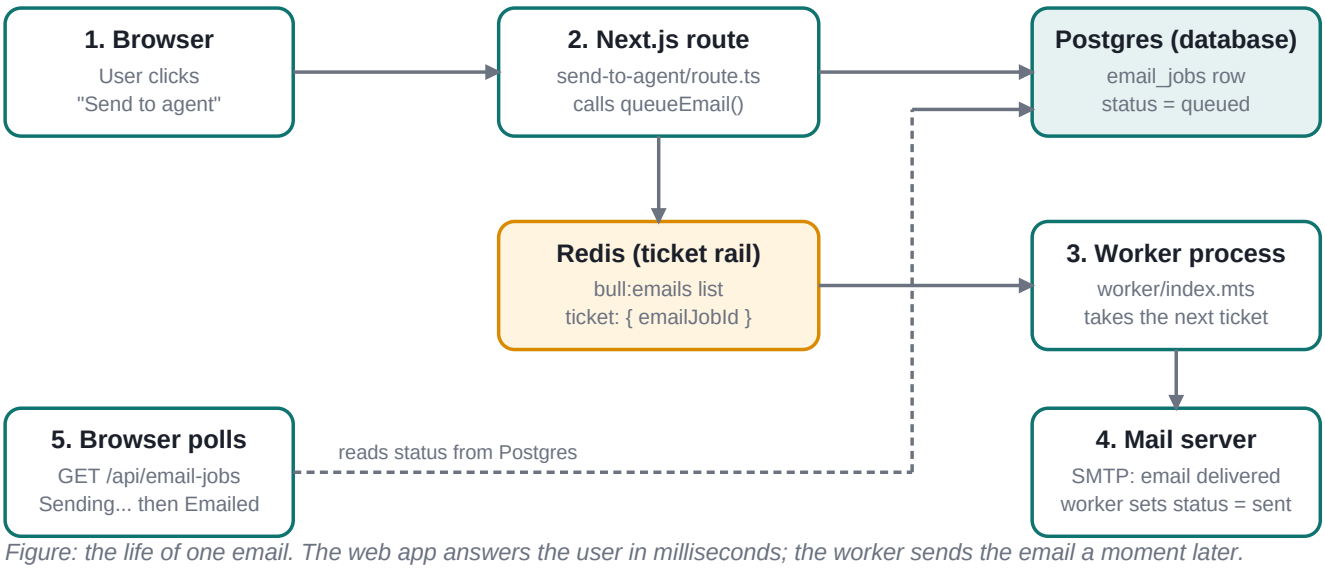

Figure: the life of one email. The web app answers the user in milliseconds; the worker sends the email a moment later.

2. What Redis is (and is not)

Redis is a very fast store that keeps data in the computer's **memory (RAM)**. It is not like Postgres:

	Postgres (database)	Redis
Shape	Tables with columns and rows	Keys that point to a value. **No tables.**
Speed	Fast, reads from disk	Extremely fast, lives in memory
Lifetime	Permanent: the source of truth	Short-lived working area. Can be lost if Redis restarts without saving

	Postgres (database)	Redis
Used here for	Clients, enquiries, orders, email_jobs, report_jobs	Only the waiting line of jobs

> **Why this is safe:** every job is also saved in Postgres (`email_jobs`, `report_jobs`) before it goes on the rail. If Redis lost its data, the record of what was supposed to happen is still in the database. Redis is the waiting line, not the filing cabinet.

3. How Redis stores the data (our real keys)

Redis has no tables. Everything is a **key** (a name) holding a **value**. The value can be a few kinds: a plain text/number (string), a list, a hash (a small bundle of named fields, like one row), a sorted set (items ordered by a number such as time). The queue library we use, **BullMQ**, creates these keys for each queue. The names below were read from the running system:

Key	Kind	What it holds
`bull:emails:id`	string	A counter: the number given to the next job (9 means nine jobs so far)
`bull:emails:1 ... :9`	hash	One job each: its name, data, options, timestamps, attempts
`bull:emails:wait`	list	Job numbers waiting for a worker. The line to the cook
`bull:emails:active`	list	Jobs a worker is working on right now
`bull:emails:delayed`	sorted set	Jobs waiting to retry later, sorted by the time to retry
`bull:emails:completed`	sorted set	Finished jobs (we keep the latest 500)
`bull:emails:failed`	sorted set	Jobs that used up all attempts (latest 500)
`bull:emails:events`	stream	Log of what happened, in order
`bull:emails:meta`	hash	Queue settings

The reports queue has exactly the same set of keys, with `bull:reports:` at the front.

What one job looks like inside Redis (a hash, our real data):

```
bull:emails:1   (hash)
  name        send
  data        {"emailJobId":1}          <- only the id, not the email text
  opts        {"attempts":3,"backoff":{"type":"exponential","delay":15000},
               "removeOnComplete":500,"removeOnFail":500}
  timestamp   1793261748107             <- when it was added
  processedOn 1793261748169             <- when the worker started it
  finishedOn  1793261753308             <- when it finished
  atm / ats   attempts made / started
```

The "table structure" of the real record lives in Postgres:

email_jobs column	Meaning
`id`	The number inside the Redis ticket (`emailJobId`)
`kind`	agent_rate_request or client_enquiry_form
`to, subject, text, html`	The finished email, already written

email_jobs column	Meaning
`logo_path`	Company logo file to attach
`status`	queued, then sent, failed or not_configured
`error, attempts`	Last SMTP error and how many tries were made
`created_at, sent_at`	When it was queued and when it was delivered

4. Which file moves data into Redis?

No Next.js file does this automatically. Next.js knows nothing about Redis. **Our own code** calls a library (BullMQ, with ioredis as the connection) at the exact moment we choose. The relevant files:

File	Job
lib/report-queue.ts	Opens the connection to Redis (`REDIS_URL`) and defines the *reports* queue. The email queue reuses its connection.
lib/email-queue.ts	**queueEmail()**: saves the email in Postgres, then **emailQueue().add(...)** puts the ticket in Redis. Also **deliverEmailJob()**, the code that really sends. If Redis cannot be reached it sends directly instead.
app/api/enquiries/[id]/send-to-agent/route.ts app/api/enquiries/send-to-client/route.ts	The routes the buttons call. They call the email functions in lib/enquiry-agent.ts and lib/enquiry-client-form.ts, which call queueEmail().
app/api/reports/jobs/route.ts	Creates a *report_jobs* row and adds the ticket to the reports queue.
worker/index.mts	**Takes tickets out** of Redis and does the work (see section 5).
app/api/email-jobs/route.ts hooks/useEmailJobs.ts	The screen asks the *database* (not Redis) every 2 seconds for the status, so it can show Sending, then Emailed.

> **The two lines that matter:** `queue.add("send", {emailJobId})` in `lib/email-queue.ts` puts a ticket on the rail. `new Worker("emails", handler)` in `worker/index.mts` takes tickets off it.

5. The worker: file or process?

Both, in two senses. `worker/index.mts` is just a **file** of code. A **worker** is that file **running** as a program, a separate process on the server, next to the web app. Starting it is the command `npm run worker`, which bundles the file and runs it with Node.

Inside that one file we create two *Worker* objects: one listening to the *reports* queue and one listening to the *emails* queue. So one running process serves both.

Can there be more than one worker?

- **Yes.** You can run several copies of the process (several containers, or several replicas). They all watch the same Redis. Redis hands each ticket to **only one** of them, so no email is sent twice.
- **Concurrency** is how many jobs one worker handles at once. We set it to **1** on purpose: emails go out one at a time so the mail server does not reject a burst (it did in testing), and a report PDF blocks the process for seconds.
- More copies = more speed, but for emails more speed is not needed and could hit the mail server's rate limit. One worker is right for now.

Do I need a second application?

In this project, yes. `npm start` only starts the web app. Nothing in it starts the worker (the file `instrumentation.ts` only records deployments). So the worker must be started separately: either a second application in Coolify with the start command `npm run worker`, or any other way of running that command continuously.

> **If you did not create a second application, the worker is not running.** Reports would stay "queued" forever and, with the new email queue, emails would stay "Sending..." forever. The one case that still works is when Redis is unreachable: the web app then sends the email directly, without a worker. Check Coolify: if there is no second app, or no app whose command is `npm run worker`, add one (steps in section 7).

6. How to confirm Redis is working

Do these in order. Each one proves one more piece.

Check	How	Good result
1. Redis is alive	Coolify > your Redis > **Terminal**, then run: `redis-cli ping` (if a password is set: `redis-cli -a PASSWORD ping`)	`PONG`
2. The app can reach it	In the web app's **Environment**, `REDIS_URL` must be the Redis' *internal* address, e.g. `redis://:PASSWORD@redis-service:6379`. Without it the app looks at `127.0.0.1` (itself) and finds nothing.	Variable is set on the **web app and the worker**
3. Jobs reach Redis	Send one email, then in the Redis terminal: `redis-cli keys 'bull:*'` `redis-cli llen bull:emails:wait`	Keys named `bull:emails:...` appear. `wait` is 0 when the worker is keeping up
4. The worker is running	Coolify > the worker app > **Logs**	`report worker ready` and `email worker ready`
5. End to end	Send to agent. The dialog shows Sending, then Emailed. Or in Postgres: `select id,status,attempts,error from email_jobs order by id desc limit 5;`	status goes **queued** then **sent**

What the symptoms mean

You see	Likely cause
Emails stay "Sending..." and `wait` keeps growing	Redis works, **the worker is not running**
Status **not_configured**	Worker ran, but `SMTP_HOST` is not set on the **worker** app
Status **failed** with a 550 / rate limit error	The mail provider is limiting you (Mailtrap testing did this). Retries wait 15s, 30s, then give up
Dialog shows the link box right away	Redis was unreachable, so the app tried to send directly (and it failed or email is off). Check `REDIS_URL`
Logo missing in the email	The worker cannot see the `uploads/` volume. Mount the same volume on both apps

7. Setting up the worker on Coolify (Hostinger VPS)

- **Redis:** if reports ever worked, you already have it. If not: Coolify > your project > **New resource > Database > Redis**. Copy its *internal* URL.
- **Database SQL:** run the four files once: `add-top-agent.sql`, `add-locations.sql`, `add-email-templates.sql`, `add-email-jobs.sql`.
- **Web app environment:** `REDIS_URL`, plus `SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, MAIL_FROM`.
- **Second application: New resource > Application**, same Git repository and branch. Set the **Start command** to `npm run worker`. No domain and no port are needed (it serves no web pages).
- **Worker environment:** the **same** variables as the web app: `DATABASE_URL, REDIS_URL, SMTP_*, MAIL_FROM, NEXTAUTH_SECRET`.
- **Storage:** mount the **same persistent volume** at `/app/uploads` on both apps (logos are read from it, report PDFs are written to it).
- **Deploy and read the worker log:** you should see *report worker ready* and *email worker ready*.

> **Good to know:** the worker and the web app are the *same code*. Every deploy of the repo should redeploy both, so the worker never runs older code than the web app.

8. Small glossary

Word	Meaning
Queue	A waiting line of jobs. We have two: *reports* and *emails*.
Job	One ticket in the line (one email, one report).
Worker	A running program that takes jobs from a queue and does them.
BullMQ	The library that manages queues on top of Redis.
Retry / backoff	If a job fails, try again later, waiting longer each time (15s, 30s ...).
Concurrency	How many jobs one worker does at the same time.
Key / hash / list	Redis ways of storing data: a name; a bundle of fields; an ordered line.